

DESARROLLO DE SISTEMAS WEB

Licenciatura en Ingeniería de Ciberseguridad e Infraestructura de Cómputo

Saberes teóricos

Unidad 2. El protocolo http

- **Unidad II. El protocolo http**
 - Paradigma solicitud-respuesta
 - Protocolos sin estado
 - Estructura de las peticiones http
 - Métodos para solicitudes http
 - Encabezados y tipos MIME
 - Códigos de respuesta
 - Evolución del protocolo http

Fuentes de información

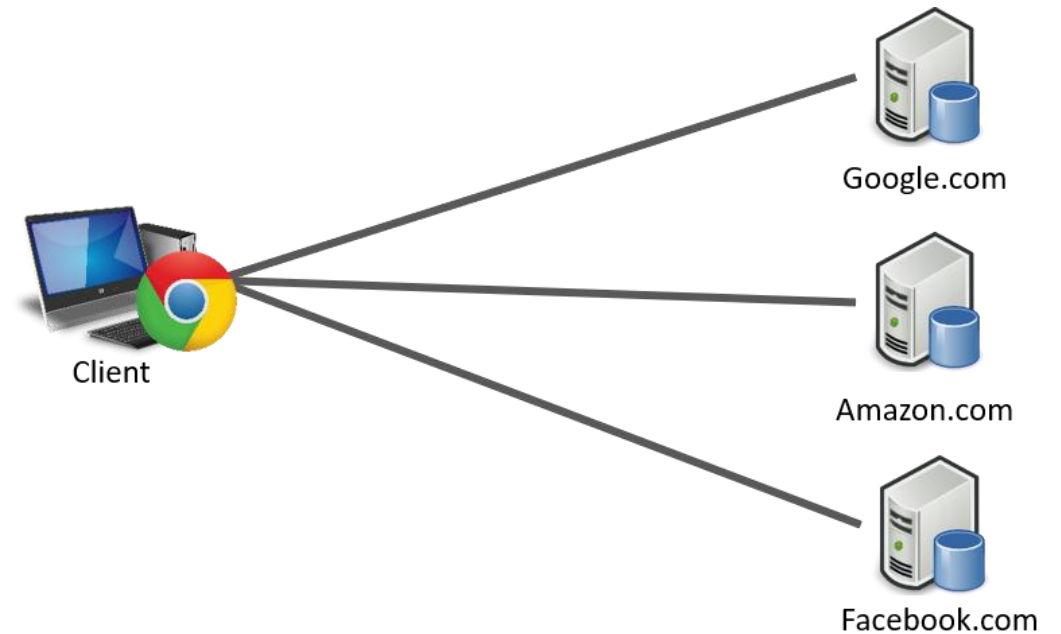
Unidad 2. El protocolo http

- Shklar Leon y Rosen Richard. Web Application, Principles, Protocols and Practices. Wiley. 2009.
- Gasston, Peter. The Modern Web: Multi-Device Web Development with HTML5, CSS3, and JavaScript. No Starch Press. 2013.
- Jennifer Niederst Robbins. Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics. O'REILLY. 2012.

HTTP

Unidad 2. El protocolo http

- Hypertext Transfer Protocol (HTTP) (o Protocolo de Transferencia de Hipertexto en español) es un protocolo de la capa de aplicación para la transmisión de documentos hipermedia, como HTML.
- Fue diseñado para la comunicación entre los navegadores y servidores web, aunque se puede utilizar para otros propósitos también.
- Sigue el clásico modelo **cliente-servidor**, en el que un cliente establece una conexión con el servidor, realiza una petición y espera hasta que recibe una respuesta del mismo.
- Se trata de un protocolo sin estado, lo cual significa que el servidor no guarda ningún dato (estado) entre dos peticiones.



Características clave del protocolo HTTP

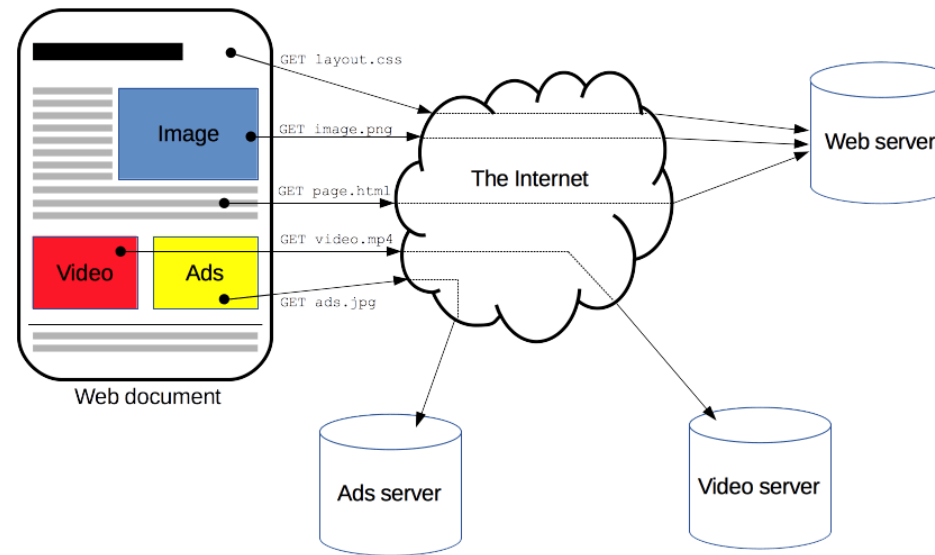
Unidad 2. El protocolo http

- **HTTP es sencillo.** esta pensado y desarrollado para ser leído y fácilmente interpretado por las personas.
- **HTTP es extensible.** las cabeceras de HTTP, han hecho que este protocolo sea fácil de ampliar y de experimentar con él.
- **HTTP es un protocolo con sesiones,** pero sin estados. HTTP es un protocolo sin estado, es decir: no guarda ningún dato entre dos peticiones en la misma sesión. Las cookies permiten crear un contexto común para cada sesión de comunicación.
- **HTTP y conexiones.** Una conexión se gestiona al nivel de la capa de transporte, y por tanto queda fuera del alcance del protocolo HTTP.
 - HTTP/1.0, HTTP/1.1, HTTP/2.

Conexiones

Unidad 2. El protocolo http

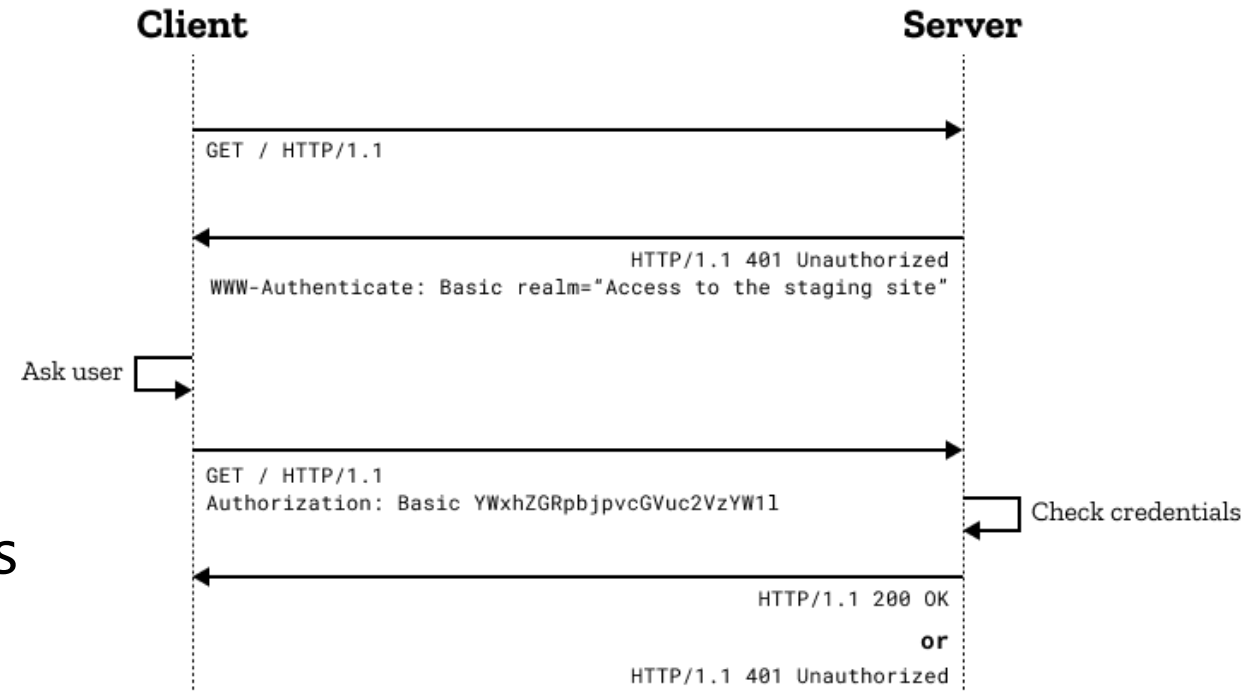
- Utiliza el puerto 80.
- Una página web completa resulta de la unión de distintos sub-documentos recibidos, como, por ejemplo: un documento que especifique el estilo de maquetación de la página web (CSS), el texto, las imágenes, vídeos, scripts, etc.



HTTP Autenticación

Unidad 2. El protocolo http

- HTTP cuenta con esquemas de autenticación los cuales son requeridos para hacer peticiones.
- Ejemplos esquemas:
 - Basic (ver [RFC 7617](#), credenciales codificadas en base64).
 - Bearer (ver [RFC 6750](#), bearer tokens de acceso en recursos protegidos mediante OAuth 2.0).
 - Digest (ver [RFC 7616](#), MD5 solo soportado en Firefox, ver Error 472823 en Firefox para encriptado SHA).



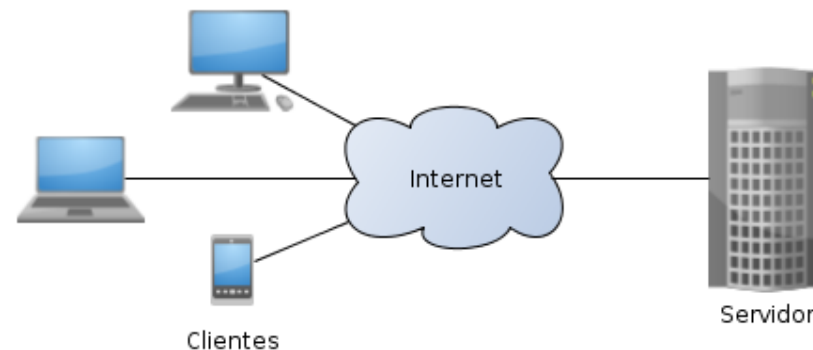
Paradigma solicitud-respuesta

Unidad 2. El protocolo http

Arquitectura Web básica: Cliente-Servidor

Unidad 2. El protocolo http

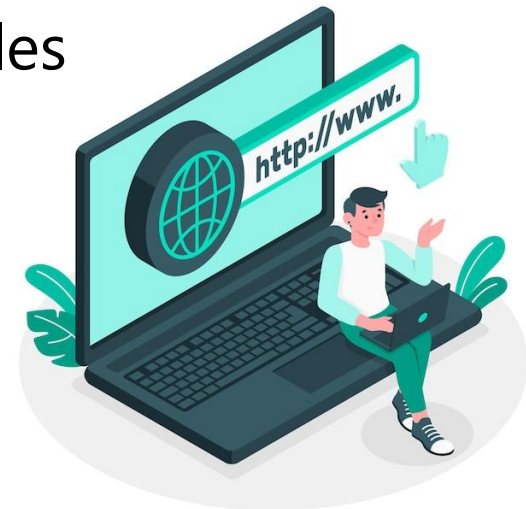
- La arquitectura **cliente-servidor** es un modelo de diseño de software en el que las tareas se reparten entre los proveedores de recursos o servicios, llamados servidores, y los demandantes, llamados clientes.
- Un cliente realiza peticiones a otro programa, el servidor, quien le da respuesta.
- La separación entre cliente y servidor es **una separación de tipo lógico**, donde el servidor no se ejecuta necesariamente sobre una sola máquina ni es necesariamente un solo programa.



Arquitectura Web básica: Cliente-Servidor

Unidad 2. El protocolo http

- **Cliente.** El remitente de una solicitud es conocido como cliente. Sus características son:
 - Es quien inicia solicitudes o peticiones, tiene por tanto un papel activo en la comunicación (dispositivo maestro o amo).
 - Espera y recibe las respuestas del servidor.
 - Por lo general, puede conectarse a varios servidores a la vez.
 - Normalmente interactúa directamente con los usuarios finales mediante una interfaz gráfica de usuario.



Arquitectura Web básica: Cliente-Servidor

Unidad 2. El protocolo http

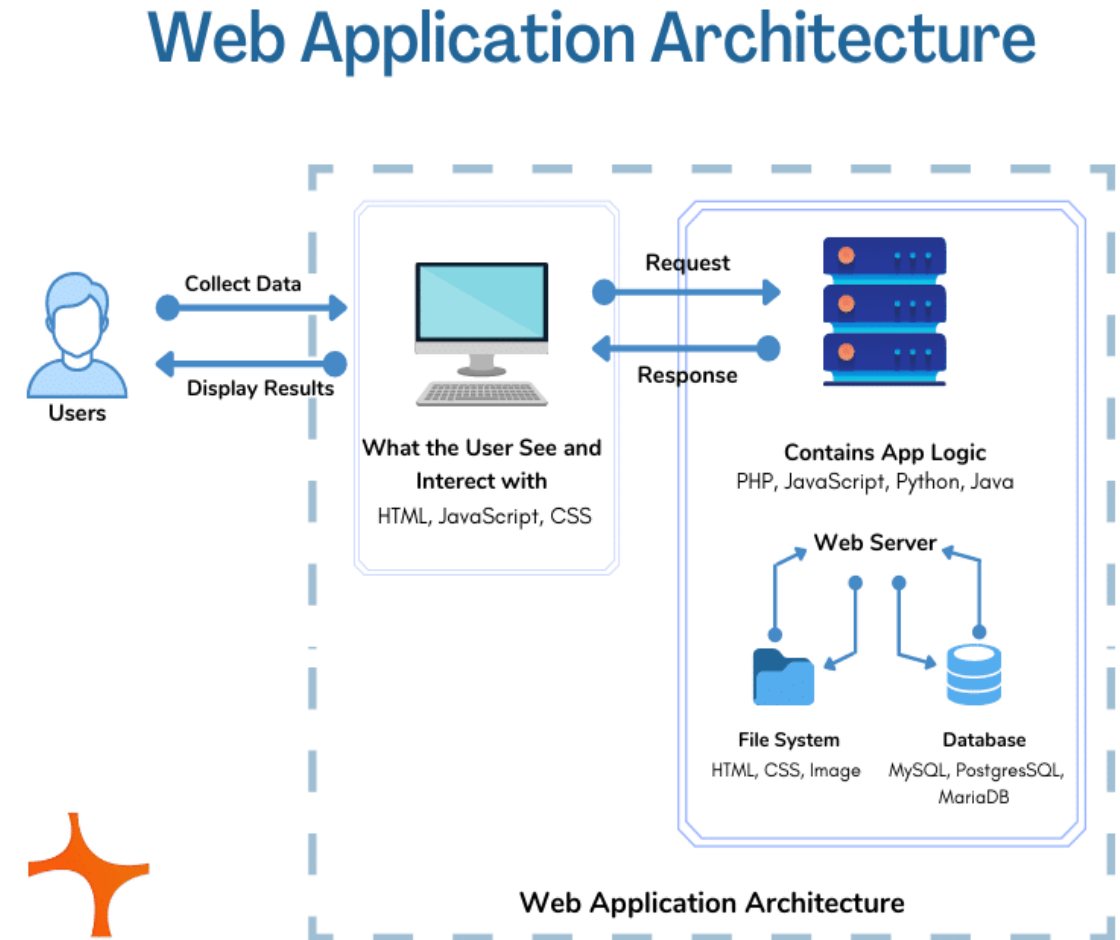
- **Servidor.** Al receptor de la solicitud enviada por el cliente se le conoce como servidor. Sus características son:
 - Al iniciarse espera a que lleguen las solicitudes de los clientes, desempeña entonces un papel pasivo en la comunicación (dispositivo esclavo).
 - Tras la recepción de una solicitud, la procesa y luego envía la respuesta al cliente.
 - Por lo general, acepta las conexiones de un gran número de clientes (en ciertos casos el número máximo de peticiones puede estar limitado).



Arquitectura Web básica: Cliente-Servidor

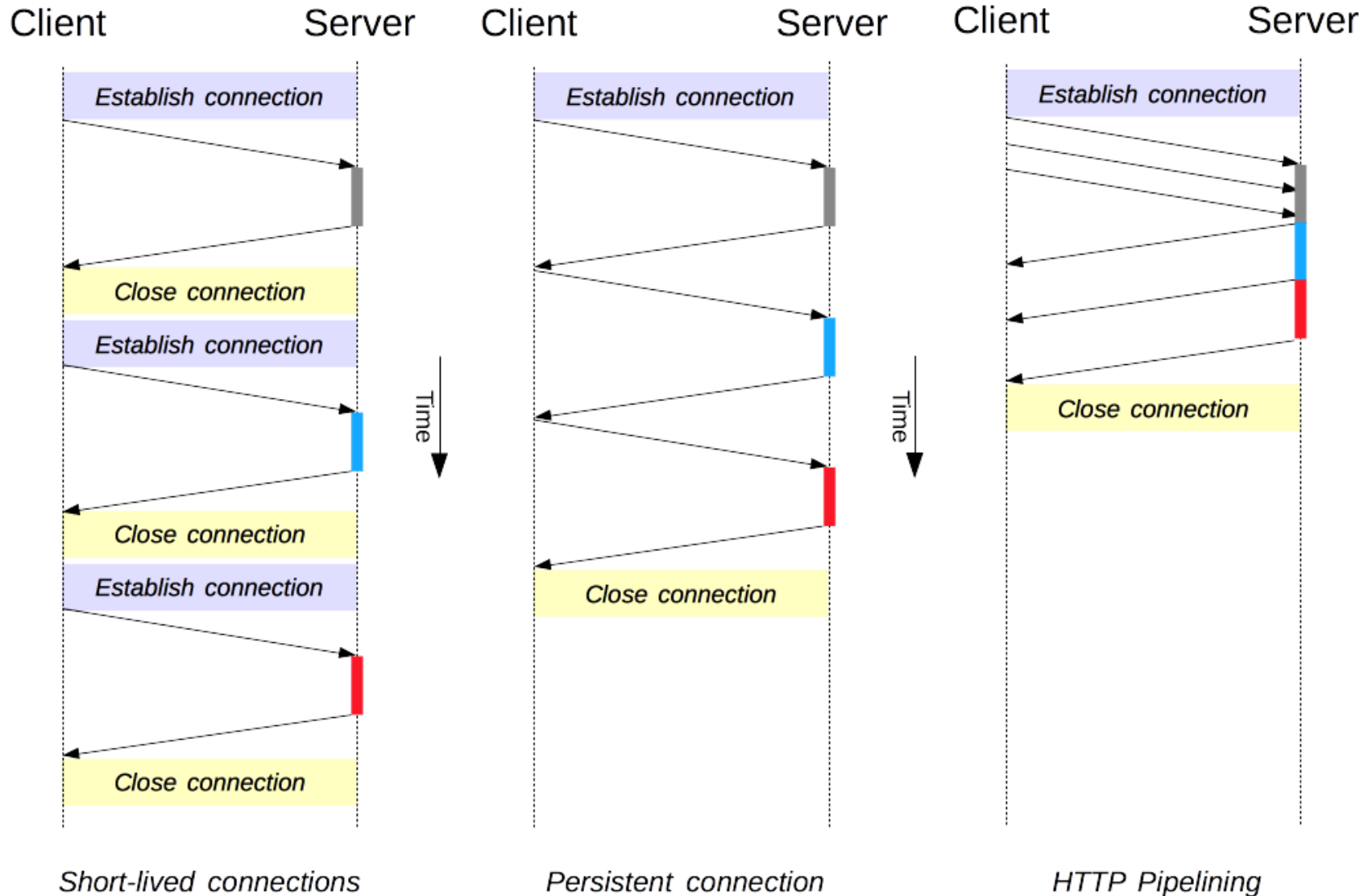
Unidad 2. El protocolo http

- 1) El usuario final utiliza la interfaz de la aplicación o el navegador y envía el comando al servidor a través de Internet.
- 2) El servidor web luego transfiere el comando al servidor solicitado.
- 3) El servidor solicitado busca los resultados de los comandos dados.
- 4) La información procesada se transfiere a la aplicación web que la envía al servidor web.
- 5) Ahora el servidor web proporciona los datos solicitados al usuario.



Arquitectura Web básica: Cliente-Servidor

Unidad 2. El protocolo http



```
GNU nano 6.2 /etc/apache2/apache2.conf
# Timeout: The number of seconds before receives and sends time out.
#
Timeout 300
#
# KeepAlive: Whether or not to allow persistent connections (more than
# one request per connection). Set to "Off" to deactivate.
#
KeepAlive On
#
# MaxKeepAliveRequests: The maximum number of requests to allow
# during a persistent connection. Set to 0 to allow an unlimited amount.
# We recommend you leave this number high, for maximum performance.
#
MaxKeepAliveRequests 100
#
# KeepAliveTimeout: Number of seconds to wait for the next request from the
# same client on the same connection.
#
KeepAliveTimeout 5

# These need to be set in /etc/apache2/envvars
User ${APACHE_RUN_USER}
Group ${APACHE_RUN_GROUP}

#
# HostnameLookups: Log the names of clients or just their IP addresses
# e.g., www.apache.org (on) or 204.62.129.132 (off).
# The default is off because it'd be overall better for the net if people
```

Encabezados de respuesta HTTP

Utilice esta característica para configurar encabezados HTTP para agregarlos a respuestas del servidor w

Agrupar por: Sin agrupar

Nombre	Valor	Tipo de entrada
X-Powered-By	ASP.NET	Hereditaria

Establecer encabezados de respuesta HTTP personalizados

☒ Habilitar HTTP keep-alive

☐ Expirar contenido web:

☐ Inmediatamente

☐ Después de:

1 días

☐ El (hora universal coordinada (UTC)):

martes, 19 de septiembre de 2023 12:00:00 a. m.

Aceptar Cancelar

Protocolos sin estado

Unidad 2. El protocolo http

HTTP sesiones

Unidad 2. El protocolo http

- **HTTP es un protocolo sin estado**, esto quiere decir que el servidor no retiene información o un estatus de los usuarios durante múltiples peticiones .
- Una sesión HTTP solo incluye una interacción petición-respuesta, la cual incluye a su vez una sesión TCP .
- Sin embargo, algunas aplicaciones Web implementan estados o sesiones del **lado del servidor** utilizando tecnologías como **cookies** o variables ocultas dentro de formularios para almacenar información de seguimiento de usuarios.

Implicaciones de Seguridad del Protocolo HTTP

Unidad 2. El protocolo http

HTTP no mantiene estado.

Las aplicaciones usan:

Cookies

Tokens

Ataque: Session Hijacking

Si la cookie de sesión viaja sin HTTPS

Cookie: PHPSESSID=298zf09hf012fh2

El atacante:

Captura la cookie.

La inserta manualmente en su navegador.

Accede como la víctima.

Implica: Secuestro de cuenta.

Cómo se evita

✓ Usar HTTPS

✓ Configurar cookies seguras:

Set-Cookie:

Secure;

HttpOnly;

SameSite=Strict

•**Secure** → solo viaja por HTTPS

•**HttpOnly** → no accesible por JavaScript

•**SameSite** → protege contra CSRF

✓ Regenerar sesión tras login

✓ Tiempo de expiración corto

✓ Invalidar sesión en logout

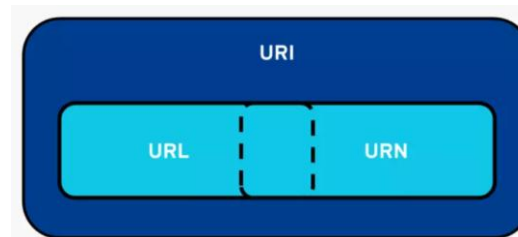
Estructura de las peticiones http

Unidad 2. El protocolo http

URI

Unidad 2. El protocolo http

- Un **identificador uniforme de recursos** o **URI** (Uniform Resource Identifier) es una cadena de caracteres que identifica los recursos (físicos o abstractos) de una red de forma única.
- Dependiendo de la situación, el recurso puede ser de muchos tipos: por ejemplo, un URI puede identificar tanto una página web como al remitente o al destinatario de un correo electrónico.
- Las aplicaciones utilizan este identificador único para interactuar con el recurso o consultar información sobre el mismo.
- Los URI pueden ser localizadores de recursos uniformes (URL), nombres de recursos uniformes (URN), o ambos.



`scheme :// authority path ? query # fragment`

`urn:isbn:0451450523`

`urn:uuid:6e8bc430-9c3a-11d9-9669-0800200c9a66`

URI

Unidad 2. El protocolo http

- Un URI consta de las siguientes partes:
 - **scheme** (esquema): Proporciona información sobre el protocolo utilizado.
 - **authority** (autoridad): Identifica el dominio.
 - **path** (ruta): Identificador específico del recurso. Muestra la ruta exacta al recurso. `https://www.uv.mx/(index.html)`
 - **query** (consulta): Representa la acción de consulta (clave=valor). Se inicia con el carácter `?`.
 - **fragment** (fragmento): Designa una parte del recurso principal. Se indica con el carácter `#`.
- Los dos elementos **obligatorios** que deben contener todos los identificadores son `scheme` y `path`.

```
scheme :// authority path ? query # fragment
```

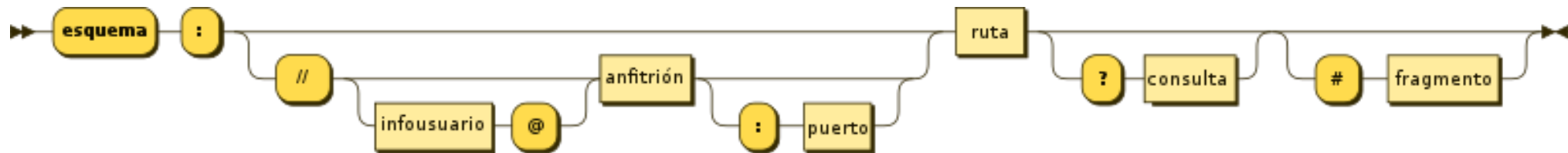
```
urn:isbn:0451450523
```

```
urn:uuid:6e8bc430-9c3a-11d9-9669-0800200c9a66
```

URI

Unidad 2 El protocolo http

- Aunque se acostumbra a llamar URL a todas las direcciones web, URI es un identificador más completo y por eso está recomendado su uso en lugar de la expresión URL.



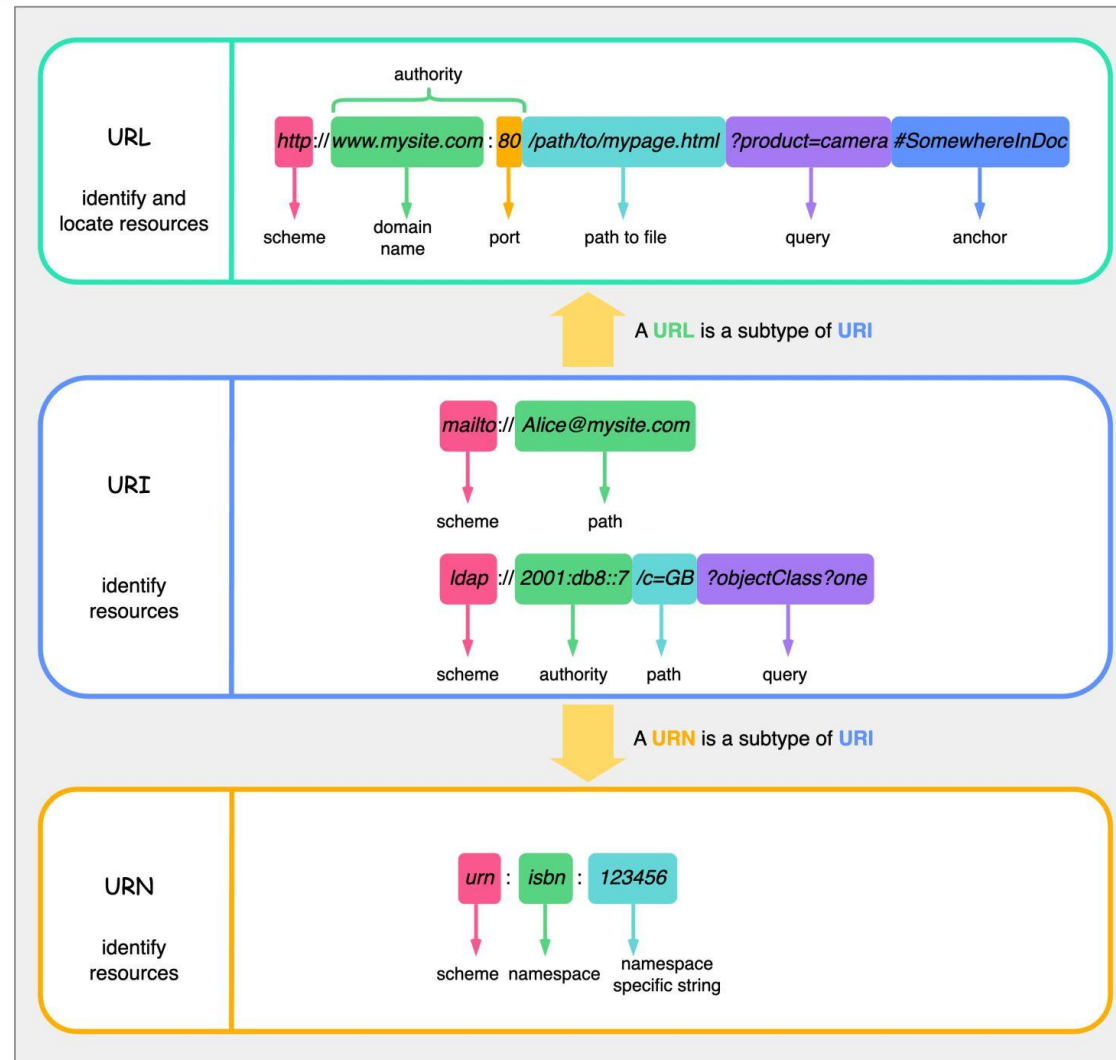
- Las siglas URI, URL y URN suelen confundirse, porque, aparte de sonar muy parecidas, se refieren a tres conceptos muy similares en términos técnicos.
- El URL se utiliza para **indicar dónde se encuentra** un recurso. Por lo tanto, también sirve para acceder a algunas páginas web por Internet.
- Por el contrario, el URN o uniform resource name es independiente de la ubicación y **designa un recurso de forma permanente**.
- Por lo tanto, si el URL se conoce principalmente como una forma de identificar un dominio web, el URN también puede tratarse de un ISBN que identifique un libro de manera indefinida.

URI y sus subtipos

Unidad 2. El protocolo http

URL vs URI vs URN

blog.bytebytego.com



URN

Unidad 2. El protocolo http

- Uniform Resource Name: nombre uniforme de recursos. Indica el nombre de un recurso que debe ser único en todo internet, además de independiente de su localización. Como el CURP de una persona. No implica la disponibilidad del recurso.
- Cada URN consta de al menos tres partes:
 - **URN**. La especificación del esquema.
 - **NID**: Namespace Identifier. Cadena predefinida como nbd (Network Block Device) o uuid (Universal Unique Identifier) o isbn (nternational Standard Book Number).
 - **NSS**: URN Resolution Server. Cadena específica del espacio de nombres que identifica con precisión el elemento.
- ISBN - Identificador único para libros.

URN:NID:NSS

urn:nbn:de:101:3-2019075675872913

urn:uuid:6r4bc420-9c3a-12i9-97d9-0665700c9a66

ISBN 1-446-2776877-40

URL

Unidad 2. El protocolo http

- Los recursos HTTP se identifican y localizan a partir de URLs (Uniform Resource Locator), las cuales tienen partes:
 - Protocolo o esquema.
 - Dominio (puede tener subdominio, dominio de alto nivel y puerto)
 - Ruta
 - Puerto
 - Parámetros (opcional)
 - Fragmento (opcional)

```
scheme :// authority path ? query # fragment
```

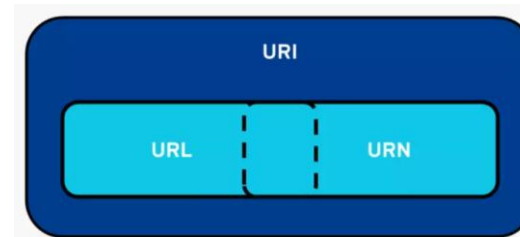
scheme (esquema): proporciona información sobre el protocolo utilizado.

authority (autoridad): identifica el dominio.

path (ruta): muestra la ruta exacta al recurso.

query (consulta): representa la acción de consulta.

fragment (fragmento): designa una parte del recurso principal.



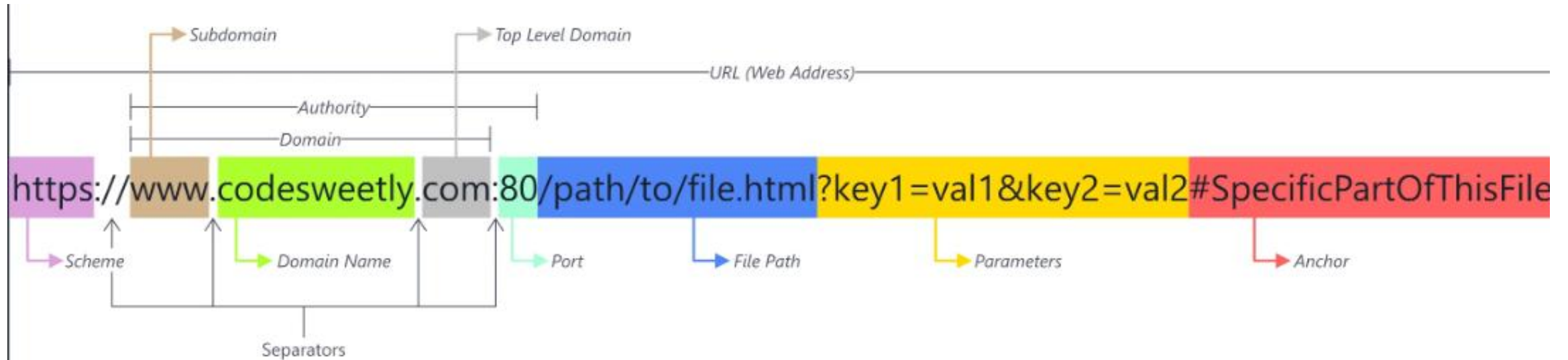
```
urn:isbn:0451450523
```

```
urn:uuid:6e8bc430-9c3a-11d9-9669-0800200c9a66
```

 <https://www.uv.mx/agenda/evento.html?area=1&evento=6803#descarga>

URL

Unidad 2. El protocolo http



`https://dsia.uv.mx/miuv/Portales/academicos/PWCGACH.ASPX?params=AC#carga`

Implicaciones de Seguridad del Protocolo HTTP

Unidad 2. El protocolo http

Manipulación de parámetros (Parameter Tampering)

`https://sitio.com/pago?monto=100`

El atacante cambia a:

`https://sitio.com/pago?monto=1`

Si el servidor no valida correctamente
puede ocurrir fraude

Cómo se evita

- ✓ Validar todo en servidor
- ✓ No confiar en valores enviados por el cliente
- ✓ Usar tokens firmados (JWT con firma válida)
- ✓ Verificar integridad con HMAC

Ejemplo

No usar:

`monto=100`

Usar:

`monto=100&firma=HMAC(monto, clave_secreta)`

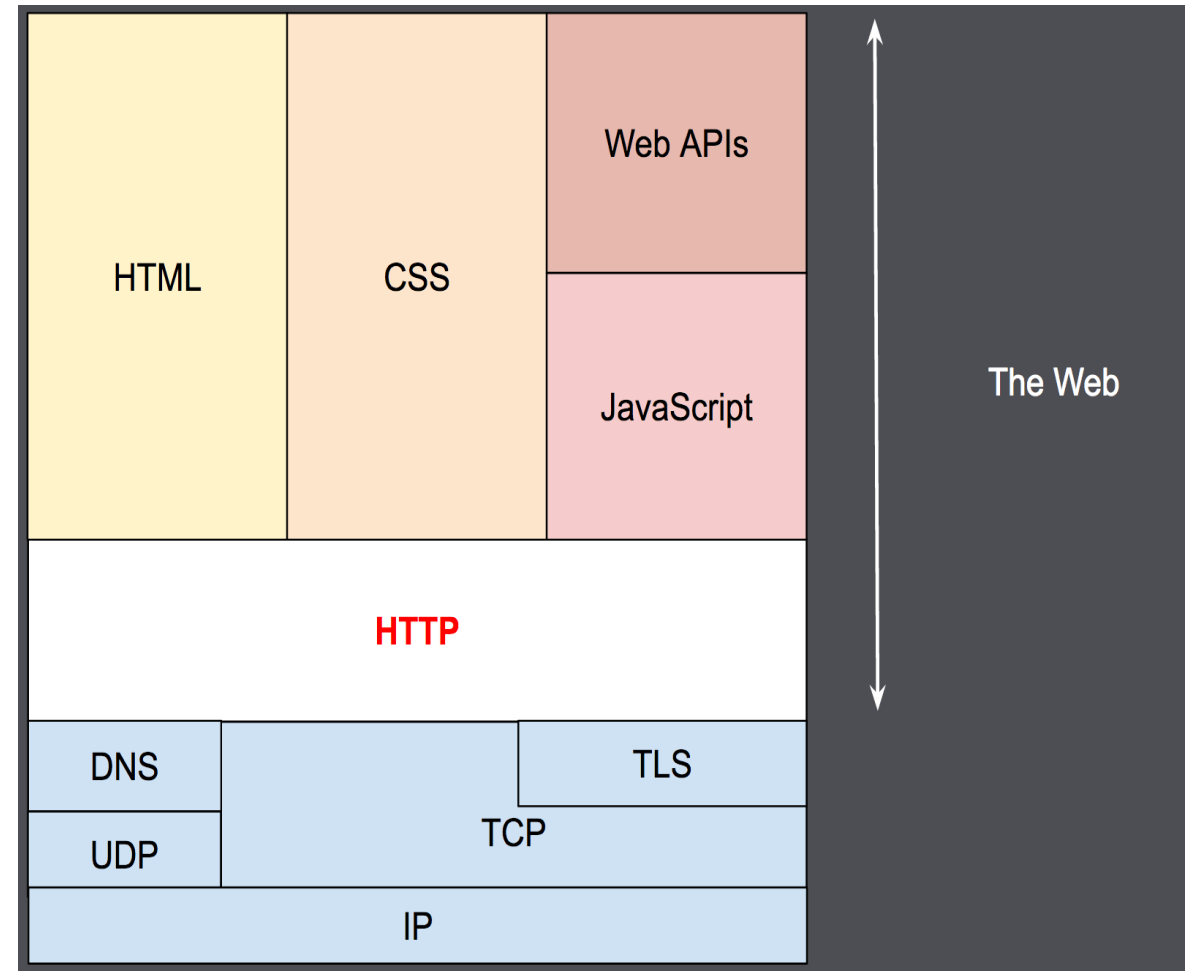
Solicitudes y respuestas HTTP

Unidad 2. El protocolo http

Peticiones y respuestas

Unidad 2. El protocolo http

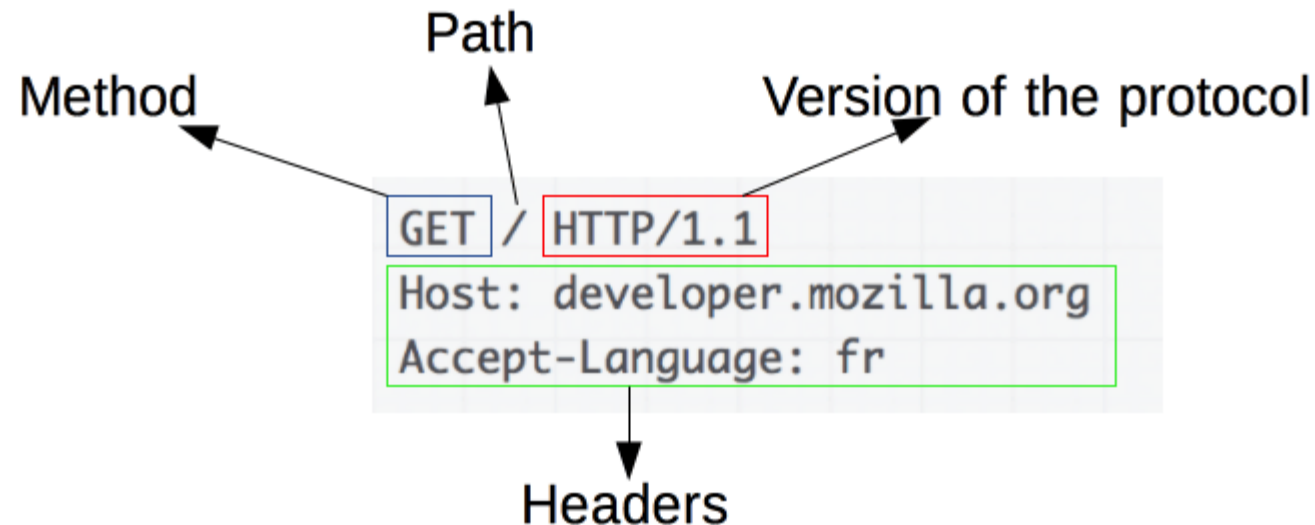
- Clientes y servidores se comunican intercambiando mensajes individuales (en contraposición a las comunicaciones que utilizan flujos continuos de datos).
- Los mensajes que envía el cliente, normalmente un navegador Web, se llaman **peticiones**, y los mensajes enviados por el servidor se llaman **respuestas**.



Mensajes HTTP

Unidad 2. El protocolo http

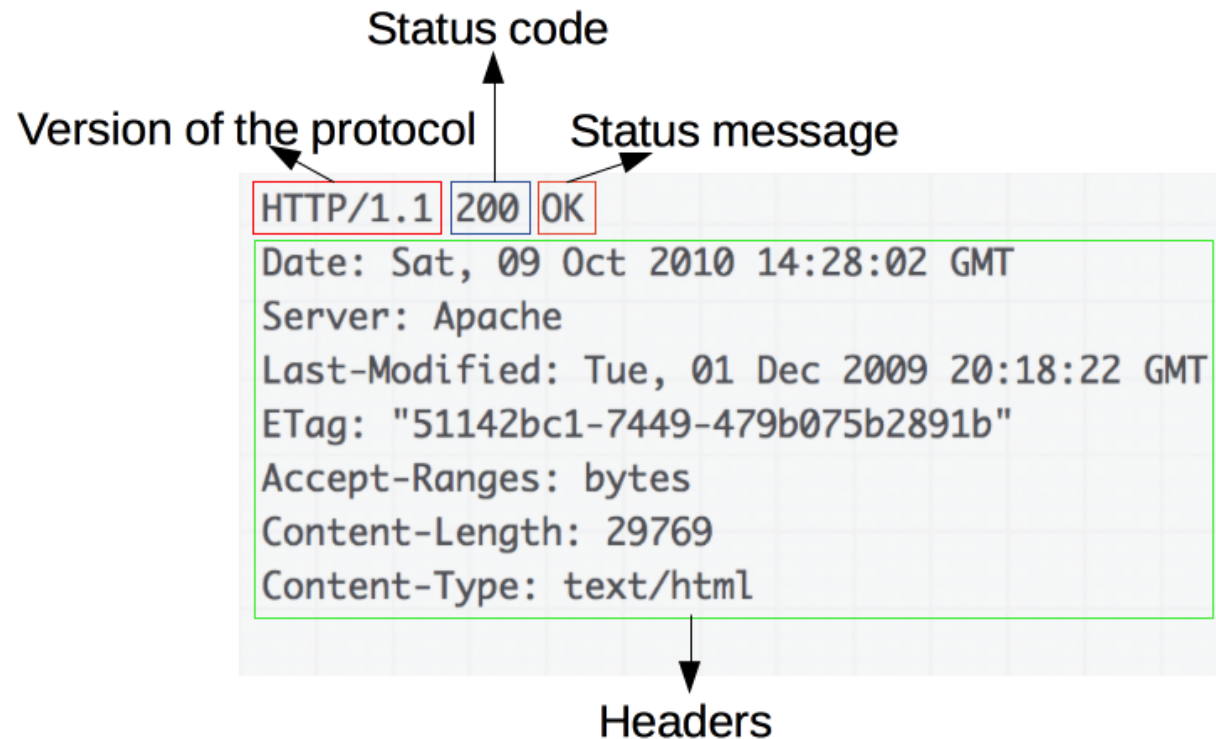
- Existen dos tipos de mensajes HTTP: peticiones y respuestas, cada uno sigue su propio formato.
- **Peticiones.** Una petición de HTTP, está formado por los siguientes campos: Un método HTTP, URL del recurso, la versión del protocolo HTTP, cabeceras HTTP opcionales o un cuerpo de mensaje.



Mensajes HTTP

Unidad 2. El protocolo http

- Las **respuestas** están formadas por los siguientes campos:
- La versión del protocolo HTTP, un código de estado, un mensaje de estado, cabeceras HTTP como las de las peticiones, opcionalmente, el recurso que se ha pedido.



Implicaciones de Seguridad del Protocolo HTTP

Unidad 2. El protocolo http

- HTTP es texto plano

Problema

HTTP transmite información en texto claro.

Si no se usa HTTPS:

- Credenciales
- Cookies de sesión
- Tokens
- Formularios
- Datos personales

pueden ser interceptados fácilmente.

Ataque: Sniffing

Ejemplo:

En una red pública (WiFi parque):

1. Un atacante ejecuta Wireshark.

2. Filtra:

http

3. Captura:

1. POST /login

username=juan&password=123456

Impacto:

Robo de credenciales.

Cómo se evita

- ✓ Usar **HTTPS (TLS 1.2 o 1.3)**
- ✓ Configurar **HSTS (Strict-Transport-Security)**
- ✓ Redirigir automáticamente HTTP → HTTPS
- ✓ Deshabilitar protocolos inseguros (SSL, TLS 1.0, 1.1)

Métodos para solicitudes http

Unidad 2. El protocolo http

Métodos de petición HTTP

Unidad 2. El protocolo http

- HTTP define un conjunto de métodos de petición para indicar la acción que se desea realizar para un recurso determinado.
- Cada uno de ellos implementan una semántica diferente.
- Están asociados a las peticiones.
- Indican la acción que se desea realizar para un recurso determinado.
- También son conocidos como verbos, HTTP verbs, (incluso si el nombre del método es un sustantivo).

Métodos de petición HTTP

Unidad 2. El protocolo http

- Los métodos pueden tener alguna de estas dos características (o ambas):
 - **Seguridad:** no tienen o no deberán tener efectos colaterales en el servidor, esto es solo regresan información y no hacen modificaciones
 - **Idempotencia:** el efecto del método sobre el servidor (los efectos colaterales) son **los mismos para exactamente la misma petición aunque esta se repita N veces**. Por definición si un método es seguro también cuenta con idempotencia.
- *En matemática y lógica, la idempotencia es la propiedad para realizar una acción determinadas veces y aun así conseguir el mismo resultado que se obtendría si se realizase una sola vez. Un elemento que cumple esta propiedad es un elemento idempotente, o un idempotente.*

Métodos de petición HTTP

Unidad 2. El protocolo http

- OPTIONS
 - GET
 - HEAD
 - POST
 - PUT
 - PATCH
 - DELETE
 - TRACE
 - CONNECT
-
- <https://developer.mozilla.org/es/docs/Web/HTTP/Methods>

OPTIONS

Unidad 2. El protocolo http

- El método OPTIONS es utilizado para describir las opciones de comunicación para el recurso de destino
- El método le permite al cliente determinar las opciones y/o requerimientos asociados con un recurso, o las capacidades del servidor, sin implicar una acción de recurso o el inicio de la recuperación de un recurso
- **Código de estado devuelto.** Tanto [200 Aceptar](#) como [204 Sin contenido](#) son códigos de estado permitidos.

Sintaxis

HTTP

OPTIONS /index.html **HTTP/1.1**

OPTIONS * HTTP/1.1

La solicitud tiene cuerpo.	No
La respuesta exitosa tiene cuerpo.	Sí
Seguro	Sí
idempotente	Sí
almacenable en caché	No
Permitido en formularios HTML	No

OPTIONS

Unidad 2. El protocolo http

cURL es la abreviatura de "Client URL" y es una herramienta de línea de comandos utilizada para la transferencia de ficheros con formato URL

INTENTO

```
curl -X OPTIONS https://example.org -i
```

```
curl -X OPTIONS https://www.uv.mx -i
```

Status: 200 (OK) Time: 222 ms Size: 0.00 kb

Content Headers (7) Raw (7) Timings

```
Allow: OPTIONS, TRACE, GET, HEAD, POST
Server: Microsoft-IIS/10.0
Public: OPTIONS, TRACE, GET, HEAD, POST
X-Powered-By: ASP.NET
Date: Sun, 17 Sep 2023 15:20:24 GMT
Content-Length: 0
```

```
C:\Users\Memo>curl --help
Usage: curl [options...] <url>
-d, --data <data>          HTTP POST data
-f, --fail                 Fail fast with no output on HTTP errors
-h, --help <category>     Get help for commands
-i, --include              Include protocol response headers in the output
-o, --output <file>        Write to file instead of stdout
-O, --remote-name          Write output to a file named as the remote file
-s, --silent              Silent mode
-T, --upload-file <file>   Transfer local FILE to destination
-u, --user <user:password> Server user and password
-A, --user-agent <name>    Send User-Agent <name> to server
-v, --verbose              Make the operation more talkative
-V, --version              Show version number and quit
```

```
curl -X OPTIONS https://sistemasfei.uv.mx -i
```

Status: 200 (OK) Time: 355 ms Size: 0.00 kb

Content Headers (8) Raw (8) Timings

```
Date: Sun, 17 Sep 2023 15:22:00 GMT
Server: Apache/2.4.41 (Ubuntu)
X-Frame-Options: SAMEORIGIN
Allow: GET,HEAD
Cache-Control: no-cache, private
Content-Length: 0
Content-Type: text/html; charset=UTF-8
```

GET

Unidad 2. El protocolo http

- El método GET solicita una representación de un recurso específico. Las peticiones que usan el método GET sólo deben recuperar datos.
- Es seguro e idempotente.
- En peticiones GET pueden agregarse parámetros en la URL.
- Por defecto, al escribir directamente una URL en el navegador se utiliza el método GET.
- **Código de estado devuelto.** Varios.

Sintaxis

```
GET /index.html
```

Petición con cuerpo	No
Respuesta válida con cuerpo	Sí
<u>Seguro</u>	Sí
<u>idempotente</u>	Sí
<u>Cacheable</u>	Sí
Permitido en HTML forms	Sí

HEAD

Unidad 2. El protocolo http

- El método HEAD pide una respuesta idéntica a la de una petición GET, pero sin el cuerpo de la respuesta.
- Una respuesta a un método HEAD no debe tener cuerpo. Si tiene uno de todos modos, ese cuerpo debe ignorarse.

Sintaxis

HTTP

HEAD /index.html

La solicitud tiene cuerpo.

No

La respuesta exitosa tiene cuerpo.

No

[Seguro](#)

Sí

[idempotente](#)

Sí

[almacenable en caché](#)

Sí

Permitido en [formularios HTML](#)

No

POST

Unidad 2. El protocolo http

- El método HTTP POST envía datos al servidor. El tipo del cuerpo de la solicitud es indicada por la cabecera Content-Type.
- La diferencia entre PUT y POST es que PUT es idempotente: llamarlo una o varias veces sucesivamente tiene el mismo efecto (no tiene efecto secundario // colateral), mientras que varios POST idénticos pueden tener efectos adicionales, como pasar una orden muchas veces.
- Como se describe en la especificación HTTP 1.1, el método POST está diseñado para permitir un método uniforme que cubra las siguientes funciones:
 - Agregar un nuevo recurso.
 - Modificación de recursos existentes.
 - Proveer un conjunto de datos, como resultado del envío de un formulario, a un proceso data-handling.

Sintaxis

```
POST /index.html
```

Pedir como cuerpo	Sí
Respuesta válida como cuerpo	Sí
Seguro	No
Idempotente	No
Cacheable	Sólo si incluye nueva información
Permitido en HTML forms (en-US)	Sí

POST

Unidad 2. El protocolo http

Un formulario simple empleando el tipo de contenido por defecto `application/x-www-form-urlencoded`:

```
HTTP

POST / HTTP/1.1
Host: foo.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 13

say=Hi&to=Mom
```

Un formulario usando el tipo de contenido `multipart/form-data`:

```
HTTP

POST /test.html HTTP/1.1
Host: example.org
Content-Type: multipart/form-data;boundary="boundary"

--boundary
Content-Disposition: form-data; name="field1"

value1
--boundary
Content-Disposition: form-data; name="field2"; filename="example.txt"

value2
```

- El tipo de contenido es seleccionado poniendo la cadena de texto adecuada (tipo MIME) en el atributo enctype o formenctype del elemento form:
- `application/x-www-form-urlencoded`: El valor por defecto. Los valores son codificados en tuplas llave-valor separadas por '&', con un '=' entre la llave y el valor.
- `multipart/form-data`: Cada valor es enviado como un dato de bloque. Para enviar archivos con input "file". Datos binarios.
- `text/plain` (HTML5).

POST

Unidad 2. El protocolo http

- Lo que realmente hará el servidor al recibir una petición POST es determinado por el propio servidor.
- Normalmente se devuelve el código 201 (Creado) y el nuevo recurso.
- Es posible que el resultado tras una acción POST no sea un recurso identificado por una URI, por lo que se suele enviar simplemente el código 200 (Ok) o el 204 (Sin contenido) o bien redirigir a otra pagina (códigos 300).
- No es seguro y no es idempotente.

```
HTTP/1.1 201 Created  
Content-Location: /nuevo.html
```

PUT

Unidad 2. El protocolo http

- La petición HTTP PUT crea un nuevo elemento o reemplaza una representación del elemento de destino con los datos de la petición.
- La diferencia entre el método PUT y el método POST es que PUT es un método **idempotente**: llamarlo una o más veces de forma sucesiva tiene el mismo efecto (sin efectos secundarios), mientras que una sucesión de peticiones POST idénticas pueden tener efectos adicionales, como enviar una orden varias veces.
- No es permitido en formularios.

Sintaxis

```
PUT /nuevo.html HTTP/1.1
```

Petición con cuerpo	Sí
Respuesta (correcta) con cuerpo	No
Seguro	No
<u>Idempotente</u>	Yes
Cacheable	No
Permitido en HTML forms (en-US)	No

PUT

Unidad 2. El protocolo http

Respuestas

Si el elemento de destino no existe y la petición `PUT` lo crea de forma satisfactoria, entonces el servidor debe informar al usuario enviando una respuesta `201` (`Created`) .

```
HTTP/1.1 201 Created
Content-Location: /nuevo.html
```

Si el elemento existe actualmente y es modificado de forma satisfactoria, entonces el servidor de origen debe enviar una respuesta `200` (`OK`) o una respuesta `204` `(en-US)` (`No Content`) para indicar que la modificación del elemento se ha realizado sin problemas.

```
HTTP/1.1 204 No Content
Content-Location: /existente.html
```

DELETE

Unidad 2. El protocolo http

- El método de solicitud HTTP DELETE elimina el recurso especificado.
- No es permitido en formularios.

Sintaxis

HTTP

```
DELETE /file.html HTTP/1.1
```

Petición con cuerpo	Puede
Respuesta válida con cuerpo	Puede
Seguro (en-US)	No
Idempotente (en-US)	Si
Se puede almacenar en Cache	No
Permitido en formularios HTML	No

DELETE

Unidad 2. El protocolo http

Respuestas

Si se aplica correctamente el método `DELETE`, hay varios códigos de estado de respuesta posibles:

- Un código de estado `202` (`Accepted`) si la acción ha sido exitosa pero aún no se ha ejecutado.
- Un código de estado `204_(en-US)` (`No Content`) si la acción se ha ejecutado y no se debe proporcionar más información.
- Un código de estado `200` (`OK`) si la acción se ha ejecutado y el mensaje de respuesta incluye una representación que describe el estado.

HTTP



HTTP/1.1 200 OK

Date: Wed, 21 Oct 2015 07:28:00 GMT

```
<html>
  <body>
    <h1>Archivo eliminado.</h1>
  </body>
</html>
```

CONNECT

Unidad 2. El protocolo http

- El método HTTP CONNECT inicia la comunicación en dos caminos con la fuente del recurso solicitado. Puede ser usado para abrir una comunicación de túnel.
- Por ejemplo, el método CONNECT puede ser usado para acceder a sitios web que usan SSL (HTTPS).
- El método CONNECT es un método de salto entre servidores

Sintaxis

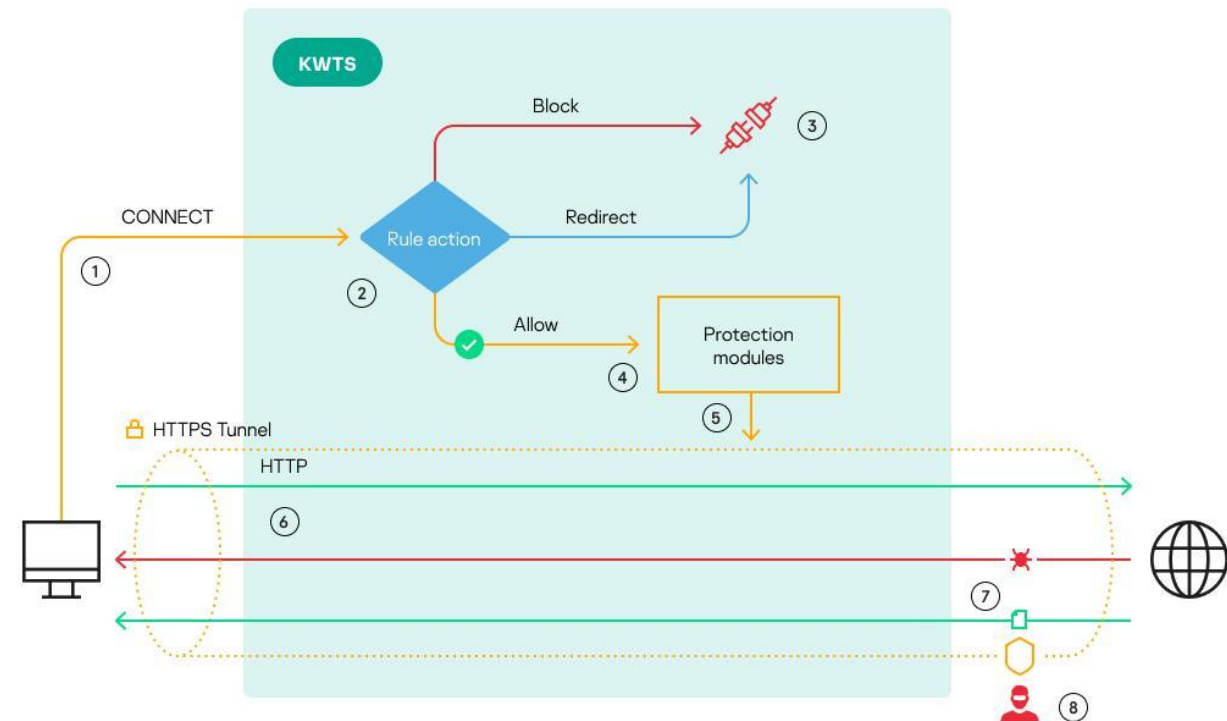
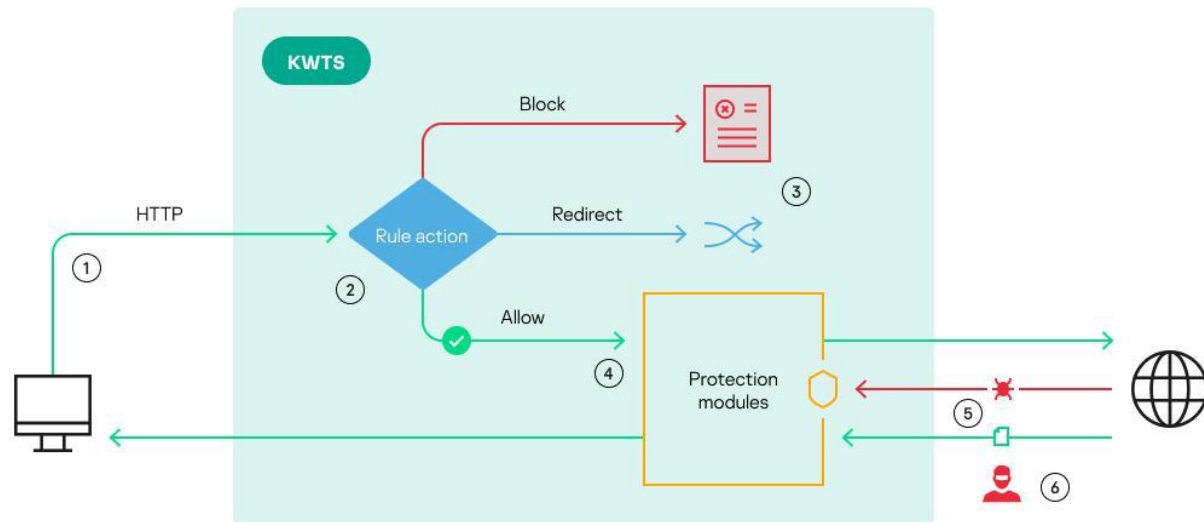
```
CONNECT www.example.com:443 HTTP/1.1
```

Contiene cuerpo la petición	No
La respuesta exitosa contiene cuerpo	Si
Seguro	No
Idempotent (en-US)	No
almacenable en caché	No
Permitido en formas HTML (en-US)	No

CONNECT

Unidad 2. El protocolo http

- El cliente realiza la petición al Servidor Proxy HTTP para establecer una conexión túnel hacia un destino deseado. Entonces el servidor Proxy procede a realizar la conexión en nombre del cliente, una vez establecida la conexión con el servidor deseado, el servidor Proxy envía los datos desde y hacia el cliente.



PATCH

Unidad 2. El protocolo http

- El método HTTP PATCH aplica modificaciones parciales a un recurso.
- El método HTTP PUT únicamente permite reemplazar completamente un documento. A diferencia de PUT, el método PATCH no es idempotente, esto quiere decir que peticiones idénticas sucesivas pueden tener efectos diferentes.
- Sin embargo, es posible emitir peticiones PATCH de tal forma que sean idempotentes.
- PATCH (al igual que POST) puede provocar efectos secundarios a otros recursos.

Sintaxis

```
PATCH /file.txt HTTP/1.1
```

Petición con cuerpo	Sí
Respuesta exitosa con cuerpo	Sí
Seguro	No
Idempotente	No
Cacheable	No
Permitido en formularios HTML (en-US)	No

PATCH

Unidad 2. El protocolo http

Petición

HTTP



```
PATCH /file.txt HTTP/1.1
Host: www.example.com
Content-Type: application/example
If-Match: "e0023aa4e"
Content-Length: 100

[description of changes]
```

Respuesta

Una respuesta exitosa es indicada con un código de respuesta [204_\(en-US\)](#), porque la respuesta no tiene mensaje en el body. (el cual tendría una respuesta con el código 200). Tenga en cuenta que también se pueden utilizar otros códigos.

```
HTTP/1.1 204 No Content
Content-Location: /file.txt
ETag: "e0023aa4f"
```

Implicaciones de Seguridad del Protocolo HTTP

Unidad 2. El protocolo http

Métodos mal protegidos

Muchos servidores permiten métodos innecesarios.

```
curl -X OPTIONS https://sitio.com -i
```

Respuesta:

```
Allow: GET, POST, PUT, DELETE
```

Si DELETE está habilitado sin autenticación adecuada puede atacarse:

Eliminación de recursos vía:

```
curl -X DELETE  
https://sitio.com/api/users/1
```

Cómo se evita

- ✓ Restringir métodos innecesarios en el servidor

Ejemplo en Apache:

```
<LimitExcept GET POST>  
    Deny from all  
</LimitExcept>
```

- ✓ Autenticación y autorización en cada endpoint
- ✓ P.e.: endpoint: DELETE /api/users/{id}

Autenticación: El sistema verifica el Token enviado. ¿Es un usuario válido? -> **Sí.**

Autorización: El sistema revisa si ese usuario tiene el rol "Admin"

- ✓ Implementar RBAC (Role-Based Access Control)
- ✓ Validación en backend (no confiar en frontend)

Implicaciones de Seguridad del Protocolo HTTP

Unidad 2. El protocolo http

HTTP permite que el cliente envíe datos arbitrarios en:

- Parámetros GET
- Cuerpo POST
- Headers
- Cookies
- JSON
- URL path

Ataque: SQL Injection

Entrada mal validada se concatena en consulta SQL.

Cómo se evita

- ✓ Usar consultas preparadas (Prepared Statements)

Ejemplo en PHP:

```
$stmt = $pdo->prepare("SELECT * FROM  
users WHERE username = ?");
```

```
$stmt->execute([$username]);
```

- ✓ ORMs (Hibernate, Entity Framework, etc.)
- ✓ Validación de entradas
- ✓ Escapar caracteres especiales
- ✓ Principio de mínimo privilegio en DB

Encabezados y tipos MIME

(Multipurpose Internet Mail Extensions)

Unidad 2. El protocolo http

HTTP headers

Unidad 2. El protocolo http

- Las cabeceras (en inglés headers) HTTP permiten al cliente y al servidor enviar información adicional junto a una petición o respuesta.
- Una cabecera de petición esta compuesta por su nombre (no sensible a las mayúsculas) seguido de dos puntos ':', y a continuación su valor (sin saltos de línea).
- Ejemplos:
 - Host: `www.uv.mx`
 - WWW-Authenticate: `Basic`
 - User-Agent: `Mozilla/5.0 (Windows NT 6.1; Win64; x64; rv:47.0) Gecko/20100101 Firefox/47.0`
 - Content-Length: `68137`
 - Content-Type: `text/html; charset=utf-8`
 - Cache-Control: `max-age=604800, must-revalidate`
 - Cookie: `PHPSESSID=298zf09hf012fh2; csrftoken=u32t4o3tb3gg43; _gat=1;`
 - Set-Cookie: `sessionId=e8bb43229de9; Domain=foo.example.com`
 - Access-Control-Allow-Origin: `https://www.uv.mx`

HTTP headers

Unidad 2. El protocolo http

Request URI: http://www.example.com

HTTP/1.1 200 OK

Content-Encoding: gzip

Age: 521648

Cache-Control: max-age=604800

Content-Type: text/html; charset=UTF-8

Date: Fri, 06 Mar 2020 17:36:11 GMT

Etag: "3147526947+gzip"

Expires: Fri, 13 Mar 2020 17:36:11 GMT

Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT

Server: ECS (dcb/7EC9)

Vary: Accept-Encoding

X-Cache: HIT

Content-Length: 648

- **HTTP/1.1** es la versión válida del protocolo HTTP.
- **200 OK** es el código de estado. Indica que el servidor ha recibido, entendido y aceptado la solicitud.
- **Content-Encoding** y **Content-Type** proporcionan información sobre el tipo de archivo.
- **Age, Cache-Control, Expires, Vary y X-Cache** se refieren al caching del archivo.
- **Etag y Last-Modified** se utilizan para el control del archivo entregado.
- **Server** se refiere al software del servidor web.
- **Content-length** es el tamaño del archivo en bytes.

Clasificación

HTTP headers

- Las Cabeceras pueden ser agrupadas de acuerdo a sus contextos:
- Autenticación.
- Almacenamiento en caché.
- Condicionales.
- Gestión de conexiones.
- Negociación de contenido.
- Cookies.
- CORS (Cross-Origin Resource Sharing).
- Descargas.
- Mensajes sobre la información del cuerpo (body).
- Proxies.
- Redirecciones.
- Contexto de petición.
- Contexto de respuesta.
- Seguridad.

Autenticación

HTTP headers

WWW-Authenticate

- Define el método de autenticación que debería ser usado para tener acceso al contenido.

Authorization

- Contiene las credenciales para autenticar a un usuario con un servidor.

Proxy-Authenticate

- Define el método de autenticación que debería ser usado para tener acceso a un recurso que se encuentre tras un servidor proxy.

Proxy-Authorization

- Contiene las credenciales para autenticar a un usuario con un servidor proxy.

Almacenamiento en caché

HTTP headers

Age

- El tiempo en el que un objeto ha estado en una caché proxy, expresado en segundos.

Cache-Control

- Especifica directivas para los mecanismos de almacenamiento en caché, tanto para peticiones como para respuestas.

Expires

- La fecha y tiempo tras las cuales una respuesta se considera agotada.

Pragma

- Cabecera específica para implementaciones que puede tener diferentes efectos a lo largo del proceso de petición-respuesta. Utilizada para implementar retrocompatibilidad con cachés de tipo HTTP/1.0 donde la cabecera Cache-Control aún no esté presente.

Warning

- Un campo de alerta general, que contiene información sobre diferentes problemas posibles.

Condicionales

HTTP headers

Last-Modified

- Se trata de un validador, indicando la fecha de la última modificación del recurso, utilizado para comparar diferentes versiones del mismo recurso. No es tan preciso como ETag, pero es más sencillo de calcular en algunos entornos. Las peticiones condicionales que usan If-Modified-Since y If-Unmodified-Since utilizan este valor para cambiar el comportamiento de la petición.

ETag

- Se trata de un validador, un tipo de hilo único identificando la versión del recurso. Las peticiones condicionales que usan If-Match y If-None-Match utilizan este valor para cambiar el comportamiento de la petición.

If-Match

- Realiza la petición condicional y aplican el método sólo si el recurso almacenado coincide con uno de los ETags asignados.

If-None-Match

- Realiza la petición condicional y aplican el método sólo si el recurso almacenado no coincide con ninguno de los ETags. Ésta cabecera se utiliza para actualizar cachés (para peticiones seguras), o para prevenir la subida de un recurso si éste ya existe en el servidor.

If-Modified-Since

- Realiza la petición condicional y espera que la entidad sea transmitida sólo si ha sido modificada tras la fecha especificada. Esta cabecera se usa para transmitir datos si la cabecera ha quedado desfasada.

If-Unmodified-Since

- Realiza la petición condicional y espera que la entidad sea transmitida sólo si no ha sido modificada tras la fecha especificada. Esta cabecera se usa para preservar la coherencia de un nuevo fragmento de un rango específico respecto a otros ya existentes, o para implementar un sistema de control de concurrencia optimista cuando se están modificando documentos existentes.

Gestión de conexiones

HTTP headers

Connection

- Controla si la conexión a la red se mantiene activa después de que la transacción en curso haya finalizado.

Keep-Alive

- Este encabezado es utilizado para enviar información acerca del acuerdo entre el cliente y servidor para utilizar una conexión persistente.
- Controla el tiempo durante el cual una conexión persistente debe permanecer abierta.

Negociación de contenido

HTTP headers

Accept

- Informa al servidor sobre los diferentes tipos de datos que pueden enviarse de vuelta. Es de tipo MIME.

Accept-Charset

- Informa al servidor sobre el set de caracteres que el cliente puede entender.

Accept-Encoding

- Informa al servidor sobre el algoritmo de codificación, habitualmente un algoritmo de compresión, que puede utilizarse sobre el recurso que se envíe de vuelta en la respuesta.

Accept-Language

- Informa al servidor sobre el language que el servidor espera recibir de vuelta. Se trata de una indicación, y no estará necesariamente sometida al control del cliente: el servidor siempre deberá estar atento para no sobrescribir una selección específica del usuario (como, por ejemplo, una selección de idiomas en una lista desplegable).

Cookies

HTTP headers

Cookie

- Contiene HTTP cookies enviadas previamente por el servidor con la cabecera Set-Cookie .

Set-Cookie

- Envía cookies desde el servidor al usuario.

Cookie2 Obsoleto

- Habitualmente contenía una cookie HTTP, enviada previamente por el servidor con la cabecera Set-Cookie2 , pero ha quedado obsoleta por la especificación. Utiliza en su lugar Cookie.

Set-Cookie2 Obsoleto

- Se utilizaba para enviar cookies desde el servidor al usuario, but has been obsoleted by the specification. pero ha quedado obsoleta por la especificación. Utiliza en su lugar Set-Cookie .

CORS (Cross-Origin Resource Sharing).

HTTP headers

Access-Control-Allow-Origin

- Indica si la respuesta puede ser compartida.

Access-Control-Allow-Credentials

- Indica si la respuesta puede quedar expuesta o no cuando el marcador de la credencial retorna como 'true'.

Access-Control-Allow-Headers

- Utilizado como respuesta a una solicitud de validación para indicár qué cabeceras HTTP pueden utilizarse a la hora de lanzar la petición.

Access-Control-Allow-Methods

- Especifica el método (o métodos) permitido al acceder al recurso, en respuesta a una petición de validación.

Access-Control-Expose-Headers

- Indica qué cabeceras pueden ser expuestas como parte de una respuesta al listar sus nombres.

Access-Control-Max-Age

- Indica durante cuánto tiempo puede guardarse el resultado de una solicitud de validación.

Access-Control-Request-Headers

- Utilizada al lanzar una petición de validación, para permitir al servidor saber qué cabeceras HTTP se utilizarán cuando la petición en cuestión se lance.

Access-Control-Request-Method

- Utilizada al enviar una solicitud de validación que permite al servidor saber qué método HTTP se utilizará cuando la petición en cuestión se lance.

Origin

- Indica el punto de origen de una petición de recogida.

Timing-Allow-Origin

- Especifica los orígenes que tienen permitido ver los valores de los atributos obtenidos mediante las características de la Resource Timing API , que de otra forma serían reportados como cero debido a los orígenes cruzados.

Descargas

HTTP headers

Content-Disposition

- Una cabecera de respuesta usada en caso de que el recurso transmitido deba mostrarse en pantalla , o debe ser gestionada como una descarga y por tanto el navegador deba presentar una pantalla de 'Guardar Como'.

Mensajes sobre la información del cuerpo (body)

HTTP headers

Content-Length

- Indica el tamaño del cuerpo del recurso, expresado en números decimales de octetos, que ha sido enviado al recipiente.

Content-Type

- Indica el tipo de medio del recurso.

Content-Encoding

- Utilizado para indicar el algoritmo de compresión.

Content-Language

- Indica el idioma (o idiomas) elegidos para los usuarios, de forma que se pueda mostrar contenido diferenciado para el usuario de acuerdo a sus preferencias de idioma.

Content-Location

- Indica un punto de origen alternativo para un recurso.

Proxies

HTTP headers

Forwarded

- Contiene información sobre el 'lado cliente' de un servidor proxy, que se alteraría o perdería si un proxy está involucrado en la ruta de la petición.

X-Forwarded-For Non-standard

- Identifica la IP de origen de un cliente que se conecta a un servidor web a través de un proxy HTTP o un equilibrador de carga.

X-Forwarded-Host Non-standard

- Identifies the la dirección original solicitada que un cliente haya utilizado para conectarse a un proxy o equilibrador de carga.

X-Forwarded-Proto Non-standard

- Identifica el protocolo (HTTP o HTTPS) que un cliente haya utilizado para conectarse a un proxy o equilibrador de carga.

Via

- Añadida por los proxies, y pueden aparecer tanto en las cabeceras de petición como las de respuesta.

Redirecciones

HTTP headers

Location

- Indica la URL a la que debe redirigir una página determinada.

Contexto de la petición

HTTP headers

From

- Contiene la dirección de email de un usuario humano que controla el gestor de peticiones.

Host

- Especifica el nombre de dominio del servidor (para alojamiento virtual), y (opcionalmente) el número de puerto TCP en el que está escuchando el servidor.

Referer

- Indica la dirección de la página web previa desde la cual un link nos ha redirigido a la actual.

Referrer-Policy

- Establece la información del referer que deberá ser enviada en las peticiones que incluyan Referer.

User-Agent

- Es utilizado para obtener información del cliente que realiza la petición HTTP. Contiene un string característico que será examinado por el protocolo de red para identificar el tipo de aplicación, sistema operativo, proveedor de software o versión del software del agente de software que realiza la petición.

Contexto de la respuesta

HTTP headers

Allow

- Lista el rango de métodos de peticiones HTTP aceptadas por un servidor.

Server

- Contiene información sobre el software utilizado por el servidor de origen para gestionar la petición.

Seguridad

HTTP headers

Content-Security-Policy (CSP)

- Controla qué recursos puede cargar el usuario para una página concreta.

Content-Security-Policy-Report-Only

- Permite a los desarrolladores web experimentar con políticas de acceso, monitorizando (pero sin implementar) sus efectos. Estos informes de violación de protocolo contienen documentos del tipo JSON enviados mediante una petición HTTP POST hacia el URI especificado.

Expect-CT

- Permite a los sitios optar por informar y/o hacer cumplir los requerimientos de Transparencia de Certificados, lo que impide que el uso de certificados publicados incorrectamente por ese sitio, pase desapercibido. Cuando un sitio habilita el encabezado Expect-CT, se solicita a Chrome que verifique que cualquier certificado para ese sitio, aparezca en los registros públicos de CT.

Public-Key-Pins (HPKP)

- Asocia una clave criptográfica pública y específica con un determinado servidor web para reducir el riesgo de MITM ataques con certificados falsificados.

Public-Key-Pins-Report-Only

- Envía reportes al report-uri especificado en la cabecera, sin bloquear la conexión entre cliente y servidor aún cuando el pinning ha sido violado.

Strict-Transport-Security (HSTS)

- Fuerza la comunicación utilizando HTTPS en lugar de HTTP.

Upgrade-Insecure-Requests

- Envía una señal al servidor expresando la preferencia del cliente por una respuesta encriptada y autenticada, y esta puede manejar de forma satisfactoria la directiva upgrade-insecure-requests .

X-Content-Type-Options

- Deshabilita el MIME sniffing y fuerza al navegador a utilizar el tipo establecido en Content-Type.

X-Frame-Options (XFO)

- Le indica al navegador que debe renderizar una página utilizando <frame>, <iframe> o <object>.

X-XSS-Protection

- Habilita los filtros de cross-site scripting.

Implicaciones de Seguridad del Protocolo HTTP

Unidad 2. El protocolo http

Headers de seguridad ausentes

Si faltan:

- X-Frame-Options
- Content-Security-Policy
- Strict-Transport-Security

Puede provocar Robo de datos vía JavaScript desde otro sitio.

Si no hay:

`X-Frame-Options: DENY`

Incrustar el sitio en un iframe invisible.

Engañar al usuario para que haga clic.

Ataque: Clickjacking

Cómo se evita

- ✓ Agregar header:

`X-Frame-Options: DENY`

`Content-Security-Policy: frame-ancestors 'none';`

Implicaciones de Seguridad del Protocolo HTTP

Unidad 2. El protocolo http

CORS mal configurado

HEADER peligroso

```
Access-Control-Allow-Origin: *  
Access-Control-Allow-Credentials:  
true
```

Esto permite:

Que cualquier dominio lea respuestas autenticadas.

Ataque:

Robo de datos vía JavaScript desde otro sitio.

Cómo se evita

✓ Especificar dominios exactos:

```
Access-Control-Allow-Origin:  
https://miapp.com
```

- ✓ No permitir credenciales innecesarias
- ✓ Validar Origin en backend

MIME Types

Unidad 2. El protocolo http

- Los MIME Types (Multipurpose Internet Mail Extensions) son la manera standard de especificar contenido a través de la red. Los tipos MIME especifican tipos de datos, como por ejemplo texto, imagen, audio, etc.
- Los navegadores a menudo usan el tipo MIME (y no la extensión de archivo) para determinar cómo procesará un documento; por lo tanto, es importante que los servidores estén configurados correctamente para adjuntar el tipo MIME correcto al encabezado del objeto de respuesta.
- Es una forma estandarizada de indicar la naturaleza y el formato de un documento.
 - MIME viene de su origen para correo electrónico donde se usa para datos adjuntos y el contenido del mensaje.
 - Se utilizan también en HTTP y HTML.

MIME Types

Unidad 2. El protocolo http

- Tipos: application, audio, image, message, multipart, text and video, font, example, and model.
- Ejemplos:
- application/json, text/plain, text/css, text/csv, text/html, text/javascript(.js), text/xml,
- application/msword (.doc), application/pdf, application/vnd.ms-excel (.xls), application/vnd.ms-powerpoint (.ppt), application/vnd.oasis.opendocument.text (.odt), application/x-www-form-urlencoded, application/xml, application/zip, audio/ogg, image/avif, image/jpeg (.jpg, .jpeg, .jfif, .pjpeg, .jpp), image/png, image/svg+xml (.svg), model/obj (.obj), multipart/form-data.

```
mime-type = type "/" [tree "."] subtype ["+" suffix]* [";" parameter];
```

https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/MIME_types/Common_types

Implicaciones de Seguridad del Protocolo HTTP

Unidad 2. El protocolo http

El navegador puede interpretar mal el contenido si no existe:

```
X-Content-Type-Options: nosniff
```

El atacante podría subir archivo .txt con código JS que el navegador ejecuta como script.

Ataque:
MIME Sniffing

Cómo se evita

✓ Agregar

```
X-Content-Type-Options: nosniff
```

✓ Configurar correctamente Content-Type

Códigos de respuesta

Unidad 2. El protocolo http

Códigos de estado de respuesta HTTP

Unidad 2. El protocolo http

- Los códigos de estado de respuesta HTTP indican si se ha completado satisfactoriamente una solicitud HTTP específica. Las respuestas se agrupan en cinco clases:
- Respuestas informativas (100–199)
- Respuestas satisfactorias (200–299)
- Redirecciones (300–399)
- Errores de los clientes (400–499)
- Errores de los servidores (500–599)
- Los códigos de estado se definen en [RFC 7231](#).
- También se les llama línea de estado.

Códigos de estado de respuesta HTTP

Unidad 2. El protocolo http

```
# Table mapping response codes to messages; entries have the
# form {code: (shortmessage, longmessage)}.
responses = {
    100: ('Continue', 'Request received, please continue'),
    101: ('Switching Protocols',
        'Switching to new protocol; obey Upgrade header'),

    200: ('OK', 'Request fulfilled, document follows'),
    201: ('Created', 'Document created, URL follows'),
    202: ('Accepted',
        'Request accepted, processing continues off-line'),
    203: ('Non-Authoritative Information', 'Request fulfilled from cache'),
    204: ('No Content', 'Request fulfilled, nothing follows'),
    205: ('Reset Content', 'Clear input form for further input.'),
    206: ('Partial Content', 'Partial content follows.'),

    300: ('Multiple Choices',
        'Object has several resources -- see URI list'),
    301: ('Moved Permanently', 'Object moved permanently -- see URI list'),
    302: ('Found', 'Object moved temporarily -- see URI list'),
    303: ('See Other', 'Object moved -- see Method and URL list'),
    304: ('Not Modified',
        'Document has not changed since given time'),
    305: ('Use Proxy',
        'You must use proxy specified in Location to access this '
        'resource.'),
    307: ('Temporary Redirect',
        'Object moved temporarily -- see URI list'),
```

Códigos de estado de respuesta HTTP

Unidad 2. El protocolo http

```
400: ('Bad Request',  
      'Bad request syntax or unsupported method'),  
401: ('Unauthorized',  
      'No permission -- see authorization schemes'),  
402: ('Payment Required',  
      'No payment -- see charging schemes'),  
403: ('Forbidden',  
      'Request forbidden -- authorization will not help'),  
404: ('Not Found', 'Nothing matches the given URI'),  
405: ('Method Not Allowed',  
      'Specified method is invalid for this server.'),  
406: ('Not Acceptable', 'URI not available in preferred format.'),  
407: ('Proxy Authentication Required', 'You must authenticate with '  
      'this proxy before proceeding.'),  
408: ('Request Timeout', 'Request timed out; try again later.'),  
409: ('Conflict', 'Request conflict.'),  
410: ('Gone',  
      'URI no longer exists and has been permanently removed.'),  
411: ('Length Required', 'Client must specify Content-Length.'),  
412: ('Precondition Failed', 'Precondition in headers is false.'),  
413: ('Request Entity Too Large', 'Entity is too large.'),  
414: ('Request-URI Too Long', 'URI is too long.'),  
415: ('Unsupported Media Type', 'Entity body in unsupported format.'),  
416: ('Requested Range Not Satisfiable',  
      'Cannot satisfy request range.'),  
417: ('Expectation Failed',  
      'Expect condition could not be satisfied.'),  
  
500: ('Internal Server Error', 'Server got itself in trouble'),  
501: ('Not Implemented',  
      'Server does not support this operation'),  
502: ('Bad Gateway', 'Invalid responses from another server/proxy.'),  
503: ('Service Unavailable',  
      'The server cannot process the request due to a high load'),  
504: ('Gateway Timeout',  
      'The gateway server did not receive a timely response'),  
505: ('HTTP Version Not Supported', 'Cannot fulfill request').
```

Evolución del protocolo http

Unidad 2. El protocolo http

Evolución del protocolo HTTP

Unidad 2. El protocolo http

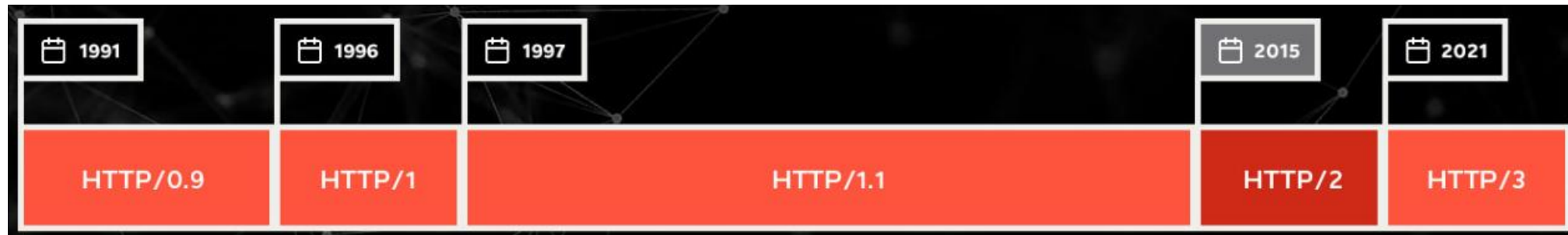
- HTTP es el protocolo en el que se basa la Web. Fue inventado por Tim Berners-Lee entre los años **1989-1991**.
- **HTTP/0.9 – El protocolo de una sola línea.** La versión inicial de HTTP, no tenía número de versión; aunque posteriormente se la denominó como 0.9 para distinguirla de las versiones siguientes.

```
GET /miPaginaWeb.html
```

La respuesta también es muy sencilla: solamente consiste el archivo pedido.

```
HTML
```

```
<html>
  Una pagina web muy sencilla
</html>
```



Evolución del protocolo HTTP

Unidad 2. El protocolo http

- **HTTP/1.0 – Desarrollando expansibilidad.** La versión del protocolo se envía con cada petición, HTTP/1.0 se añade a la línea de la petición GET.
 - Se envía también un **código de estado** al comienzo de la respuesta, permitiendo así que el navegador pueda responder al éxito o fracaso de la petición realizada, y actuar en consecuencia (como actualizar el archivo o usar la caché local de algún modo).
 - El concepto de **cabeceras de HTTP**, se presentó tanto para las peticiones como para las respuestas, permitiendo la transmisión de meta-data y conformando un protocolo muy versátil y ampliable.
 - Con el uso de las cabeceras de HTTP.

```
GET /mypage.html HTTP/1.0
User-Agent: NCSA_Mosaic/2.0 (Windows 3.1)

200 OK
Date: Tue, 15 Nov 1994 08:12:31 GMT
Server: CERN/3.0 libwww/2.17
Content-Type: text/html
<HTML>
Una pagina web con una imagen
    <IMG SRC="/miImagen.gif">
</HTML>
```

Continúa con una segunda conexión y la petición de una imagen:

```
GET /myImagen.gif HTTP/1.0
User-Agent: NCSA_Mosaic/2.0 (Windows 3.1)

200 OK
Date: Tue, 15 Nov 1994 08:12:32 GMT
Server: CERN/3.0 libwww/2.17
Content-Type: text/gif
(image content)
```

Evolución del protocolo HTTP

Unidad 2. El protocolo http

- **HTTP/1.1 – El protocolo estándar.** La primera versión estandarizada de HTTP: el protocolo HTTP/1.1, se publicó en 1997, tan solo unos meses después del HTTP/1.0.
- HTTP/1.1 aclaró ambigüedades y añadió numerosas mejoras:
 - Una **conexión podía ser reutilizada**, ahorrando así el tiempo de re-abrirla repetidas veces para mostrar los recursos empotrados dentro del documento original pedido.
 - Enrutamiento ('Pipelining' en inglés) se añadió a la especificación, permitiendo realizar una segunda petición de datos, antes de que fuera respondida la primera, disminuyendo de este modo la latencia de la comunicación.
 - Se permitió que las respuestas a peticiones, podían ser divididas en sub-partes.
 - Se añadieron controles adicionales a los mecanismos de gestión de la cache.
 - La negociación de contenido, incluyendo el lenguaje, el tipo de codificación, o tipos, se añadieron a la especificación, permitiendo que servidor y cliente, acordasen el contenido más adecuado a intercambiarse.
 - Gracias a la cabecera, Host, pudo ser posible alojar varios dominios en la misma dirección IP.

```
GET /static/img/header-background.png HTTP/1.1
Host: developer.mozilla.org
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.9; rv:50.0) Gecko/20100101 Firefox/50.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://developer.mozilla.org/es/docs/Glossary/Simple_header
```

```
200 OK
Age: 9578461
Cache-Control: public, max-age=315360000
Connection: keep-alive
Content-Length: 3077
Content-Type: image/png
Date: Thu, 31 Mar 2016 13:34:46 GMT
Last-Modified: Wed, 21 Oct 2015 18:27:50 GMT
Server: Apache
```

```
(image content of 3077 bytes)
```

Evolución del protocolo HTTP

Unidad 2. El protocolo http

- **El uso de HTTP para transmisiones seguras.** En vez de transmitir HTTP sobre la capa de TCP/IP, se creó una capa adicional sobre esta en 1994: SSL. Evolucionó hasta ser el protocolo TLS, con versiones 1.0, 1.1, 1.2, 1.3, que fueron apareciendo para resolver vulnerabilidades.
- **En el año 2000, un nuevo formato para usar HTTP fue diseñado: REST** (del inglés: 'Representational State Transfer'). Permite manipular "recursos" (datos o funcionalidades) mediante métodos estándar como GET, POST, PUT y DELETE, siendo altamente escalable y flexible.
- **Seguridad de HTTP.** Se han agregado restricciones para proteger la Web. El nivel de restricción es comunicado desde el servidor al cliente, mediante una serie de nuevas cabeceras HTTP.
- Estas están especificadas en los documentos como CORS (del inglés Cross-Origin Resource Sharing o recursos compartidos de orígenes cruzados) y el CSP (del inglés: Content Security Policy o política de seguridad de contenidos).

Evolución del protocolo HTTP

Unidad 2. El protocolo http

- **HTTP/2 – Un protocolo para un mayor rendimiento.** Estandarizado de manera oficial en Mayo de 2015, HTTP/2. El protocolo HTTP/2, tiene notables diferencias fundamentales respecto a la versión anterior HTTP/1.1
 - Es un **protocolo binario**, en contraposición a estar formado por cadenas de texto, tal y como estaban basados sus protocolos anteriores. Así pues no se puede leer directamente, ni crear manualmente. A pesar de este inconveniente, gracias a este cambio es posible utilizar en él técnicas de optimización.
 - Es un protocolo **multiplexado**. Peticiones paralelas pueden hacerse sobre la misma conexión, no está sujeto pues a mantener el orden de los mensajes, ni otras restricciones que tenían los protocolos anteriores HTTP/1.x
 - Comprime las cabeceras, ya que estas, normalmente son similares en un grupo de peticiones. Esto elimina la duplicación y retardo en los datos a transmitir.
 - Esto permite al servidor almacenar datos en la caché del cliente, previamente a que estos sean pedidos, mediante un mecanismo denominado 'server push'.

Evolución del protocolo HTTP

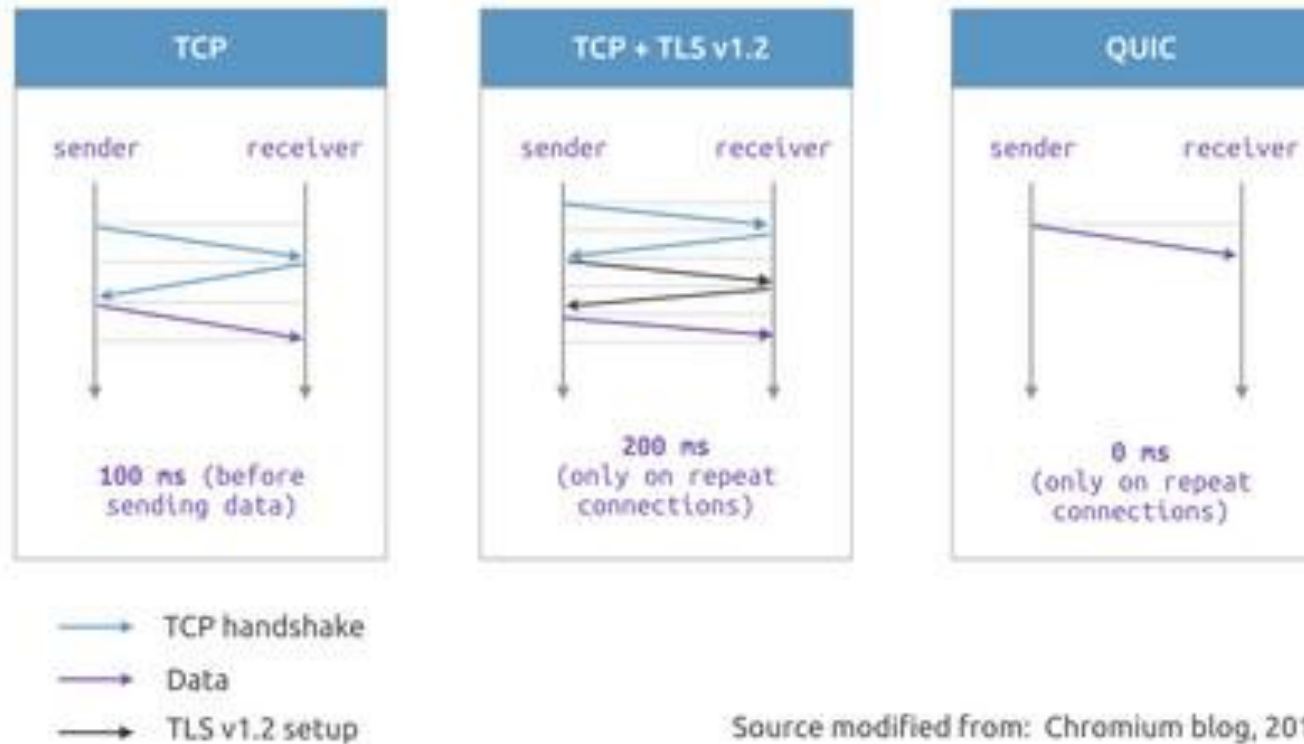
Unidad 2. El protocolo http

- **HTTP/3 – Conexiones UDP rápidas.** El sustituto para este protocolo en HTTP/3 es QUIC, siglas que significan Quick UDP Internet Connections (Conexiones UDP Rápidas en Internet).
- Está basado en otro viejo protocolo de los años 80 llamado UDP, y que a diferencia del TCP no requiere del intercambio continuo de información entre el emisor y el receptor del paquete de información.
- El protocolo de transferencia ya no se encarga de la integridad de los datos, ese peso recaerá de cada aplicación que lo use.
- Chrome lo tiene desde la build 79 lanzada en diciembre 2019, y Firefox desde su build 72 de enero de 2020.
- También Cloudflare ha implementado la compatibilidad de este protocolo.
- **El salto al HTTP/3 no se ha notado demasiado a nivel de usuario**, poco a poco se ve cómo algunos servicios adaptados empezarán a cargar un poco más rápido.

Evolución del protocolo HTTP

Unidad 2. El protocolo http

Comparison between TCP, TCP + TLS, and QUIC



Source modified from: Chromium blog, 2015

Implicaciones de Seguridad del Protocolo HTTP

Unidad 2. El protocolo http

- Principios de defensa a aplicar en HTTP:
 - Cifrar todo (HTTPS)
 - Validar todo en backend
 - Configurar headers de seguridad
 - Aplicar principio de mínimo privilegio
 - No confiar nunca en el cliente

Resumen

Unidad 2. El protocolo http

- El protocolo HTTP es un protocolo ampliable y fácil de usar.
- Su estructura cliente-servidor, junto con la capacidad para usar cabeceras, permite a este protocolo evolucionar con las nuevas y futuras aplicaciones en Internet.
- Aunque la versión del protocolo HTTP/2 añade algo de complejidad, al utilizar un formato en binario, esto aumenta su rendimiento, y la estructura y semántica de los mensajes es la misma desde la versión HTTP/1.0.
- El flujo de comunicaciones en una sesión es sencillo y puede ser fácilmente estudiado e investigado con un simple monitor de mensajes HTTP.

Preguntas de repaso

Unidad 2. El protocolo http

1. Método HTTP que aplica modificaciones parciales a un recurso.

a) GET

b) PATCH

c) HEAD

d) CONNECT

2. Son partes de alto nivel que conforman un identificador de recursos URI.

a) esquema e
identificador específico
del recurso

b) GET, POST

c) esquema, código de
estado

d) métodos, códigos de
estado y verbos

3. Son códigos de estado de respuestas satisfactorias.

a) 100-199

b) 200-299

c) 300-399

d) 400-499

Preguntas de repaso

Unidad 2. El protocolo http

4. Son códigos de estado de errores de los clientes.

a) 100-199

b) 200-299

c) 300-399

d) 400-499

5. Son los dos tipos de mensajes HTTP.

a) GET y POST

b) Peticiones y respuestas

c) SSL y TLS

d) OPTIONS y GET

6. Utiliza conexiones UDP en lugar de TCP.

a) HTTP/1.0

b) HTTP/1.1

c) HTTP/2.0

d) HTTP/3.0

Preguntas de repaso

Unidad 2. El protocolo http

7. Este encabezado contiene el nombre de dominio cualificado del URI que se solicita.

a) Host

b) Content-Length

c) Content-Type

d) User-agent

8. Es otra forma de llamar a las pequeñas operaciones que pueden ser realizadas sobre un recurso utilizando el protocolo HTTP.

a) Host

b) Métodos

c) Encabezados

d) Líneas o códigos de estados

9. Este encabezado es utilizado para obtener información del cliente que realiza la petición HTTP.

a) Host

b) Content-Length

c) Content-Type

d) User-agent



Preguntas